Atividade: Criar loops

Introdução

Como analista de segurança, algumas das medidas que você toma para proteger um sistema envolvem repetição. Por exemplo, você pode precisar investigar vários endereços IP que tentaram se conectar à rede. Em Python, instruções iterativas podem ajudar a automatizar processos repetitivos como esses, tornando-os mais eficientes.

Neste laboratório, você praticará a escrita de instruções iterativas em Python.

Dicas para concluir este laboratório

Cenário

Você trabalha como analista de segurança e escreve programas em Python para automatizar a exibição de mensagens sobre tentativas de conexão de rede, detectar endereços IP que estão tentando acessar dados restritos e gerar números de identificação de funcionários para o departamento de vendas.

Tarefa 1

Nesta tarefa, você criará um loop relacionado à conexão a uma rede.

Escreva uma instrução iterativa que exiba o resultado `Connection could not be established` três vezes. Use a `for` palavra-chave `if`, a `range()` função `while` e uma variável de loop do tipo `while` `i`. Certifique-se de substituir o código `### YOUR CODE HERE ###` existente pelo seu próprio código antes de executar a célula seguinte.

Em [3]:
```
# Instrução iterativa usando `for`, `range()` e uma variável de loop `i`
# Exibe "Não foi possível estabelecer conexão." três vezes

para i em range ( 3 ):
    print ( "Não foi possível estabelecer conexão." )
```

```
Não foi possível estabelecer conexão.
Não foi possível estabelecer conexão.
Não foi possível estabelecer conexão.
```

Dica 1

Dica 2

Tarefa 2

A `range()` função também pode receber uma variável. Para repetir uma ação específica um determinado número de vezes, você pode primeiro atribuir um valor inteiro a uma variável. Em seguida, você pode passar essa variável para a `range()` função dentro de um `for` laço.

In your code that displays a network message connection, incorporate a variable called `connection_attempts` . Assign the positive integer of your choice as the value of that variable and fill in the missing variable in the iterative statement. Be sure to replace the `### YOUR CODE HERE ###` with your own code before you run the following cell. Test out the code with different values for `connection_attempts` and observe what happens.

In [6]:
```
# Create a variable called `connection_attempts` that stores the number of
times the user has tried to connect to the network

connection_attempts = 5

# Iterative statement using `for`, `range()`, a loop variable of `i`, and `
connection_attempts`
# Display "Connection could not be established." as many times as specified
by `connection_attempts`

for i in range(connection_attempts):
    print("Connection could not be established")
```

```
Connection could not be established
Connection could not be established
Connection could not be established
Connection could not be established
Connection could not be established
```

Hint 1

Hint 2

Task 3

This task can also be achieved with a `while` loop. Complete the `while` loop with the correct code to instruct it to display `"Connection could not be established."` three times.

In this task, a `for` loop and a `while` loop will produce similar results, but each is based on a different approach. (In other words, the underlying logic is different in each.) A `for` loop terminates after a certain number of iterations have completed, whereas a `while` loop terminates once it reaches a certain condition. In situations where you do not know how many times the specified action should be repeated, `while` loops are most appropriate.

Be sure to replace the `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [7]:
```
# Assign `connection_attempts` to an initial value of 0, to keep track of h
ow many times the user has tried to connect to the network

connection_attempts = 0

# Iterative statement using `while` and `connection_attempts`
# Display "Connection could not be established." every iteration, until con
nection_attempts reaches a specified number

while connection_attempts < 3:
    print ("Connection could not be established.")

    # Update `connection_attempts` (increment it by 1 at the end of each it
eration)
    connection_attempts = connection_attempts + 1
```

```
Connection could not be established.
Connection could not be established.
Connection could not be established.
```

Hint 1

Hint 2

Hint 3

Question 1

What do you observe about the differences between the `for` loop and the `while` loop that you wrote?

A principal diferença é que o loop for é ideal para repetições com um número definido de vezes, enquanto o loop while é mais adequado para repetir uma ação até que uma condição específica deixe de ser verdadeira.

Task 4

Now, you'll move onto your next task. You'll automate checking whether IP addresses are part of an allow list. You will start with a list of IP addresses from which users have tried to log in, stored in a variable called `ip_addresses`. Write a `for` loop that displays the elements of this list one at a time. Use `i` as the loop variable in the `for` loop.

Be sure to replace the `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [9]:
```
# Assign `ip_addresses` to a list of IP addresses from which users have tri
ed to log in

ip_addresses = ["192.168.142.245", "192.168.109.50", "192.168.86.232", "19
2.168.131.147",
                "192.168.205.12", "192.168.200.48"]

# For loop that displays the elements of `ip_addresses` one at a time

for i in ip_addresses:
    print(i)
```

```
192.168.142.245
192.168.109.50
192.168.86.232
192.168.131.147
192.168.205.12
192.168.200.48
```

Hint 1

Hint 2

Task 5

You are now given a list of IP addresses that are allowed to log in, stored in a variable called `allow_list`. Write an `if` statement inside of the `for` loop. For each IP address in the list of IP addresses from which users have tried to log in, display `"IP address is allowed"` if it is among the allowed addresses and display `"IP address is not allowed"` otherwise.

Be sure to replace the `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [11]:
```
# Assign `allow_list` to a list of IP addresses that are allowed to log in

allow_list = ["192.168.243.140", "192.168.205.12", "192.168.151.162", "192.
168.178.71",
              "192.168.86.232", "192.168.3.24", "192.168.170.243", "192.16
8.119.173"]

# Assign `ip_addresses` to a list of IP addresses from which users have tri
ed to log in

ip_addresses = ["192.168.142.245", "192.168.109.50", "192.168.86.232", "19
2.168.131.147",
                "192.168.205.12", "192.168.200.48"]

# For each IP address in the list of IP addresses from which users have tri
ed to log in,
# If it is among the allowed addresses, then display “IP address is allowe
d”
# Otherwise, display “IP address is not allowed”

for i in ip_addresses:
        if i in allow_list:
             print("IP address is allowed")
        else:
                print("IP address is not allowed")
```

```
IP address is not allowed
IP address is not allowed
IP address is allowed
IP address is not allowed
IP address is allowed
IP address is not allowed
```

Hint 1

Hint 2

Hint 3

Task 6

Imagine now that the information the users are trying to access is restricted, and if an IP address outside the list of allowed IP addresses attempts access, the loop should terminate because further investigation would be needed to assess whether this activity poses a threat. To achieve this, use the `break` keyword and expand the message that is displayed to the user when their IP address is not in `allow_list` to provide more specifics. Instead of `"IP address is not allowed"` , display `"IP address is not allowed. Further investigation of login activity required"` .

Be sure to replace the `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [19]:
```
# Assign `allow_list` to a list of IP addresses that are allowed to log in

allow_list = ["192.168.243.140", "192.168.205.12", "192.168.151.162", "192.
168.178.71",
              "192.168.86.232", "192.168.3.24", "192.168.170.243", "192.16
8.119.173"]

# Assign `ip_addresses` to a list of IP addresses from which users have tri
ed to log in

ip_addresses = ["192.168.142.245", "192.168.109.50", "192.168.86.232", "19
2.168.131.147",
                "192.168.205.12", "192.168.200.48"]

# For each IP address in the list of IP addresses from which users have tri
ed to log in,
# If it is among the allowed addresses, then display “IP address is allowe
d”
# Otherwise, display “IP address is not allowed”

for i in ip_addresses:
        if i in allow_list:
                print("IP address is not allowed")
        else:
                print("IP address is not allowed. Further investigation of
login activity required")
                break
```

```
IP address is not allowed. Further investigation of login activity require
d
```

Hint 1

Hint 2

Hint 3

Task 7

You'll now complete another task. This involves automating the creation of new employee IDs.

You have been asked to create employee IDs for a Sales department, with the criteria that the employee IDs should all be numbers that are unique, divisible by 5, and falling between 5000 and 5150. The employee IDs can include both 5000 and 5150.

Write a `while` loop that generates unique employee IDs for the Sales department by iterating through numbers and displays each ID created.

Be sure to replace the `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [20]:
```
# Assign the loop variable `i` to an initial value of 5000

i = 5000

# While loop that generates unique employee IDs for the Sales department by
iterating through numbers
# and displays each ID created

while i <= 5150:
    print(i)
    i = i + 5
```

```
5000
5005
5010
5015
5020
5025
5030
5035
5040
5045
5050
5055
5060
5065
5070
5075
5080
5085
5090
5095
5100
5105
5110
5115
5120
5125
5130
5135
5140
5145
5150
```

Hint 1

Hint 2

Hint 3

Task 8

You would like to incorporate a message that displays `Only 10 valid employee ids remaining` as a helpful alert once the loop variable reaches `5100` .

To do so, include an `if` statement in your code.

Be sure to replace the `### YOUR CODE HERE ###` with your own code before you run the following cell.

In [25]:
```
# Assign the loop variable `i` to an initial value of 5000

i = 5000

# While loop that generates unique employee IDs for the Sales department by
iterating through numbers
# and displays each ID created
# This loop displays "Only 10 valid employee ids remaining" once `i` reache
s 5100

while i <= 5150:
    print(i)
    if i == 5100:
        print("Only 10 valid employee ids remaining")
    i = i + 5
```

```
5000
5005
5010
5015
5020
5025
5030
5035
5040
5045
5050
5055
5060
5065
5070
5075
5080
5085
5090
5095
5100
Only 10 valid employee ids remaining
5105
5110
5115
5120
5125
5130
5135
5140
5145
5150
```

Hint 1

Hint 2

Hint 3

Question 2

Why do you think the statement `print(i)` is written before the conditional rather than inside the conditional?

A instrução print(i) fica fora da condicional para que todos os IDs sejam exibidos na sequência, enquanto o if serve apenas para inserir um alerta específico em um momento determinado do loop.

Conclusion

What are your key takeaways from this lab?

Os loops permitem automatizar tarefas repetitivas de segurança, enquanto condicionais e comandos de interrupção como o break oferecem o controle necessário para validar dados e interromper processos suspeitos instantaneamente.

```
In [ ]:
```